SSU Data Dash 2026: *Duo Sucker*

Preston Cai, Lorance Jiang

Video (YouTube): `https://youtu.be/YOUR-VIDEO`
Code and submission files (GitHub): `https://github.com/Preston-Cai/SSU-Data-Dash`

Summary

We built separate ensembles for off-market and list-aware valuation. Each combines boosted trees using property and strictly historical local-sale features with a TF–IDF text model that reads listing remarks. In six-month walk-forward validation (December 2025–May 2026), the current rerun produced the results in Table 1. Our main finding is that carefully timed local-sale history matters more than changing tree libraries: a neighbourhood feature is useful only when it is constructed from sales already known on the prediction date.

1 Data and features

We used physical attributes (area midpoints, lot size, bedrooms, bathrooms, parking, style, and property type), municipality and community, rounded latitude/longitude cells, calendar and seasonal variables, and indicators extracted from remarks. Local-history features summarize log price and log price per square foot for community/type/style and geographic groups. The list-aware track additionally uses list price and historical sold-to-list ratios; the off-market pipeline never receives list price. We excluded the external AVM and fields known only after sale. No outside data were used.

We also supplied the trees with log area and log lot area, area-per-bedroom and bathrooms-per-bedroom ratios, garage/parking indicators, condominium flags, multi-resolution geographic cells, and community–property-type and community–style interactions. Remarks became compact renovation, condition, luxury, transit, basement, and "as-is" indicators; the separate Ridge model kept the full word-level remarks rather than reducing them to flags.

The targets were chosen to match the metric: off-market models predict log(sold_price), while list-aware models predict log(sold_price/list_price). This reduces the influence of the largest homes and makes the list-aware task a percentage adjustment rather than a second full-price estimate. Local statistics are smoothed toward the overall market when a group is rare. We retained both ordinary encodings and recency-weighted encodings because the former estimates a stable local level, while the latter can respond to market drift. The final text training window is limited to the latest 36 months, and the strongest list-aware recency encoder uses a four-month half-life.

2 Models

The off-market prediction is a calibrated geometric blend of recency-aware LightGBM (32%), word TF–IDF Ridge (19%), and XGBoost (49%). The list-aware blend uses ordinary target-encoded LightGBM

(11.3%), recency-aware LightGBM (37.7%), an independent off-market value anchor (15.5%), and a smoothed historical community-plus-property-type sold-to-list median (35.5%). Trees capture nonlinear interactions among structure, location, and earlier comparable sales; Ridge contributes a different view through listing language. We blend them because their errors are not identical, even when a component is weaker by itself. The specialized list-aware anchor helps decide how much to trust an asking price by comparing it with both prior local sale ratios for the same property family and a valuation that never saw list price.

We also tested a dense PyTorch MLP with standardized numeric features, property/style indicators, and learned municipality/community embeddings. It reached 11.19% off-market and 5.63% list-aware MdAPE, so we did not promote it. Its feature set lacked remarks, richer interactions, and the historical local price signals that most helped the tree ensemble; this particular architecture also fit mixed tabular structure less effectively than boosted trees.

The screening experiments explain the final model choice. On the November screen, leakage-safe local encodings reduced off-market MdAPE from 8.86% to 7.75%, recency weighting reached about 7.69%, and XGBoost reached 7.62%. The components were retained because their errors were complementary. In contrast, CatBoost reached 10.34% off-market MdAPE, a direct comparable estimator reached 11.83%, and a local-anchor residual model reached about 7.85%. These alternatives were therefore rejected rather than included for complexity alone. A weighted geometric blend was selected because the models operate on log-price targets; a multiplicative calibration then removes a remaining systematic bias.

3 Validation

We used walk-forward validation rather than a random split. For each month from December 2025 through May 2026, every component was rebuilt using only sales strictly before that month's cutoff, then evaluated on that month. Target encodings are also chronological *within* the training set: a training home's local statistics and global smoothing prior use only sales with an earlier `date_sold`. Homes sold on the same date are processed as a batch and cannot reveal their targets to one another. Validation homes use the complete permitted pre-cutoff history, and their sold prices never enter feature construction. Blend weights and calibration were fitted on September–November 2025. The reported list-aware result is the conservative 3.98% score from the pre-specified walk-forward evaluation; later post-validation tuning is not used as evidence for the final result. Across the six official months, monthly MdAPE ranged from 7.05% to 7.67% off-market and from 3.87% to 4.37% list-aware; May was the weakest month for both tracks, consistent with market drift rather than a single anomalous row.

4 Results

Track	Homes	MdAPE % (95% CI)	MAPE % (95% CI)	Median signed error %
off-market	10,069	7.27 (7.11–7.45)	9.81 (9.62–10.00)	-1.06
list-aware	10,069	3.98 (3.86–4.05)	5.15 (5.06–5.24)	-0.94

Table 1: Walk-forward validation on the listings. MdAPE: median absolute percentage error. MAPE: mean absolute percentage error. Brackets: bootstrap 95% interval.

The table is regenerated directly from row-level predictions by `python run.py report`. MdAPE describes the typical percentage miss, while MAPE exposes larger errors. We report both because a model can improve the median while leaving an unacceptable tail. The rerun is the authoritative score:

earlier experimental numbers used leave-one-out encodings that did not meet the handout's stricter "earlier sales only" rule and are not claimed here.

The saved final artifact contains the fitted encoders, component models, blend weights, calibration, and submission files. Reloading that checkpoint reproduces the predictions byte-for-byte, and both CSVs pass the starter submission checker. This separates the reproducible submission model from the exploratory screening scripts.

5 Where the model fails

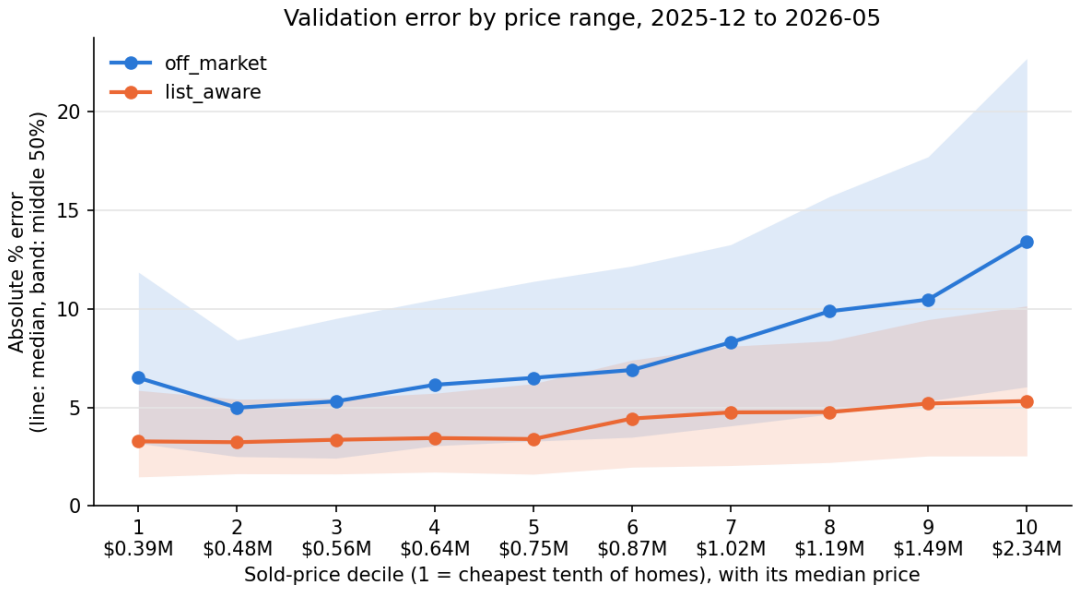


Figure 1: Absolute percentage error by sold-price decile.

Errors rise sharply in the expensive tail: the highest sold-price decile is about 13% MdAPE off-market versus about 6–7% in the cheapest deciles, while list-aware error rises to about 5% from roughly 3%. Sparse communities and rare property combinations also fail more often because their historical encodings are pulled toward the market prior. The off-market task is harder because it lacks the seller's asking price; the month-by-month range above also shows sensitivity to market drift. Text helps when remarks reveal renovation or condition, but synthetic or vague descriptions add little. These patterns suggest calibrated uncertainty by price and local-history support, not just a single point estimate.

6 Limitations

The June–September hidden test remains the only truly unseen evaluation, so walk-forward improvement is evidence rather than a guarantee. Rare groups are still noisy despite smoothing, and recency weighting cannot anticipate a sudden market shift. The ensemble is also slower and harder to audit than a single model. December–May was inspected during development, so future tuning should use nested time splits or a newly reserved period. Next we would add segment-level uncertainty intervals and test whether simpler components can be removed without sacrificing accuracy.

Outside data

Source	URL	Periods covered
None		

Reproducing our results

```
pip install -r gbt_ensemble_experiments/requirements-attempts.txt
python gbt_ensemble_experiments/validate.py
python run.py report
python gbt_ensemble_experiments/train_final.py
python run.py check
```